

FPGA DESIGN

Memory Game

Project Report

Submitted by

MUHAMMED SADIQUE P K

Government Engineering College Thrissur

Abstract

The project “*Memory Game*” is implemented on the Zybo Z7-10 development board using the Zynq-7000 SoC architecture. The system utilizes the Processing System (PS) and Programmable Logic (PL) interface to create an interactive memory-based game. The hardware platform was designed using Vivado IP Integrator, while the game logic was implemented in C language using Vitis Unified IDE.

The objective of the project is to test and improve the player’s memory by generating a sequence of LED patterns that must be reproduced correctly using push buttons. The game progressively increases in difficulty by extending the sequence length after each successful round. Communication between the processor and hardware peripherals is achieved through AXI GPIO interfaces.

The project demonstrates the practical application of embedded systems design, AXI-based communication, processor-programmable logic integration, GPIO interfacing, and real-time user interaction on FPGA-based systems.

Chapter 1

Introduction

Memory games are widely used to improve concentration, memory retention, and reaction speed. The player is required to observe a sequence of visual indications and reproduce them correctly.

In this project, a Memory Game is implemented on the Zybo Z7-10 board using the Zynq-7000 Processing System and Programmable Logic architecture. LEDs are used to display randomly generated patterns, while push buttons are used for user inputs. The game begins with a single pattern and gradually increases the sequence length after each successful attempt.

The hardware platform is developed in Vivado using AXI GPIO peripherals connected to the Zynq Processing System. The application software is written in C and executed on the ARM Cortex-A9 processor through Vitis.

The project provides practical experience in:

- Zynq PS-PL Interface Design
- AXI GPIO Communication
- Embedded C Programming
- Hardware-Software Co-Design
- Real-Time System Development
- FPGA-Based Embedded Systems

Chapter 2

Methodology

2.1 System Design

The Memory Game architecture consists of:

- Zynq-7000 Processing System
- AXI GPIO for LEDs
- AXI GPIO for Push Buttons
- AXI GPIO for Switches
- AXI SmartConnect
- Processor System Reset Module

The hardware platform was designed using Vivado Block Design and exported to Vitis for software development.

2.2 Hardware Implementation

The hardware design was created using Vivado IP Integrator. Three AXI GPIO peripherals were connected to the Processing System:

- LED GPIO (Output)
- Button GPIO (Input)
- Switch GPIO (Input)

Communication between the Processing System and GPIO peripherals is performed through the AXI bus.

2.3 Software Implementation

The game logic was implemented in C using Vitis Unified IDE.

The software performs:

- Random sequence generation
- LED sequence display
- Button input detection
- User response verification
- Level progression
- Success and Game Over indication

2.4 Development Tools

Table 2.1: Development Tools

Tool	Purpose
Vivado 2025.2	Hardware Design
Vitis Unified IDE 2025.2	Software Development
Zybo Z7-10 Board	Hardware Platform
AXI GPIO IP	Peripheral Interface
Embedded C	Application Development

Chapter 3

Block Diagram

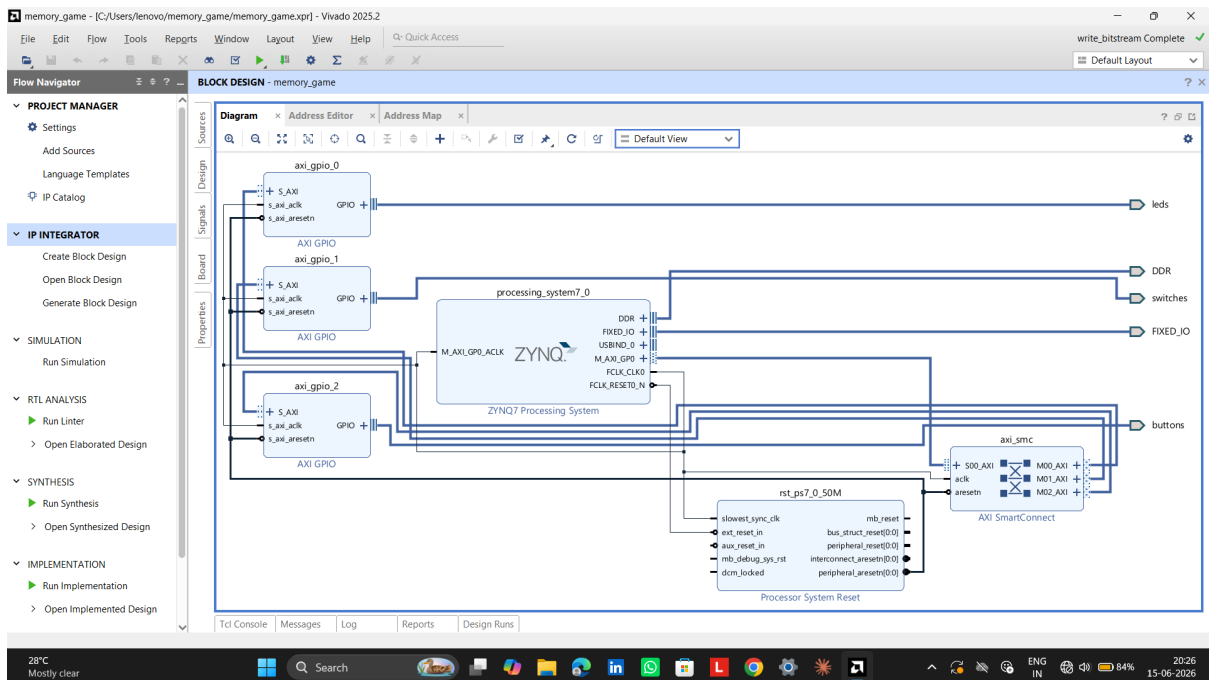


Figure 3.1: Vivado Block Design of Memory Game System

3.1 Description

The Processing System communicates with the LEDs, Buttons, and Switches through AXI GPIO modules. AXI SmartConnect is used to route AXI transactions between the ARM processor and peripherals. The Processor System Reset module generates synchronized reset signals for the design.

Chapter 4

Working Model

4.1 Game Operation

The Memory Game operates as follows:

4.1.1 Step 1: Game Initialization

When the system starts, all hardware peripherals are initialized through AXI GPIO drivers. The LEDs, buttons, and switches are configured with appropriate input/output directions.

4.1.2 Step 2: Start Indication

At the beginning of a new game, all four LEDs flash three times to indicate that the game is ready.

4.1.3 Step 3: Sequence Generation

A random number generator creates a sequence consisting of values between 0 and 3. Each value corresponds to one LED.

Initially, the sequence contains only one LED pattern.

4.1.4 Step 4: Sequence Display

The generated sequence is displayed to the player using LEDs.

Each LED remains ON for approximately 0.5 seconds followed by a short OFF interval before displaying the next pattern.

4.1.5 Step 5: User Input

After the sequence display is completed, the player must reproduce the sequence using the push buttons.

Each button corresponds to a specific LED position.

When a button is pressed:

- The corresponding LED glows briefly.
- The input is compared with the stored sequence.

4.1.6 Step 6: Verification

The entered sequence is checked against the generated sequence.

Correct Input

If all button presses match the sequence:

- The player advances to the next level.
- Another random pattern is appended.
- The sequence length increases by one.
- All LEDs flash twice as a success indication.

Wrong Input

If any button press is incorrect:

- The game immediately ends.
- All LEDs remain ON for two seconds.
- The game resets and waits for a new start cycle.

4.1.7 Step 7: Winning Condition

The player continues until the maximum sequence length is reached. Successfully completing all rounds results in victory and demonstrates complete memory retention of the generated pattern sequence.

4.2 Video Demonstration

https://drive.google.com/drive/folders/1w6B6QcLV01qrtE3lDOM1a79p90TSBzCF?usp=drive_link

Chapter 5

Software Implementation

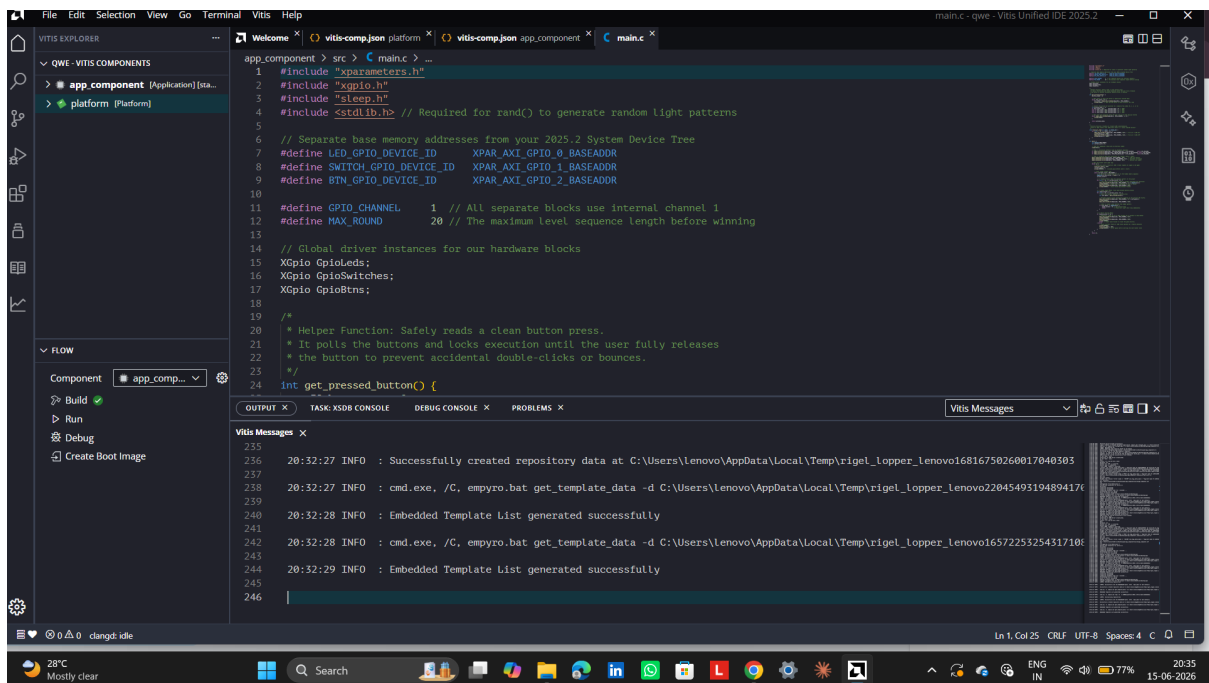


Figure 5.1: Memory Game Application Developed in Vitis

The application software is written in Embedded C and executed on the ARM Cortex-A9 processor. AXI GPIO drivers are used to communicate with LEDs, switches, and push buttons connected through the Programmable Logic.

Chapter 6

Source Code

The Memory Game application was developed in Embedded C using Vitis Unified IDE. The complete source code is shown below.

```
1 #include "xparameters.h"
2 #include "xgpio.h"
3 #include "sleep.h"
4 #include <stdlib.h> // Required for rand() to generate random
   light patterns
5
6 // Separate base memory addresses from your 2025.2 System Device
   Tree
7 #define LED_GPIO_DEVICE_ID      XPAR_AXI_GPIO_0_BASEADDR
8 #define SWITCH_GPIO_DEVICE_ID   XPAR_AXI_GPIO_1_BASEADDR
9 #define BTN_GPIO_DEVICE_ID      XPAR_AXI_GPIO_2_BASEADDR
10
11 #define GPIO_CHANNEL            1 // All separate blocks use internal
   channel 1
12 #define MAX_ROUND               20 // The maximum level sequence length
   before winning
13
14 // Global driver instances for our hardware blocks
15 XGpio GpioLeds;
16 XGpio GpioSwitches;
17 XGpio GpioBtns;
18
19 /*
20  * Helper Function: Safely reads a clean button press.
21  * It polls the buttons and locks execution until the user fully
   releases
22  * the button to prevent accidental double-clicks or bounces.
```

```

23  */
24  int get_pressed_button() {
25      u32 btn_state = 0;
26
27      // Wait until at least one button is physically pressed down
28      while (btn_state == 0) {
29          btn_state = XGpio_DiscreteRead(&GpioBtns, GPIO_CHANNEL);
30          usleep(10000); // 10ms polling delay to ease processor
                        // bus load
31      }
32
33      // Convert the active bit position to a simple array index
        (0, 1, 2, or 3)
34      int pressed_index = -1;
35      if (btn_state & 0x1)      pressed_index = 0; // BTN0
36      else if (btn_state & 0x2) pressed_index = 1; // BTN1
37      else if (btn_state & 0x4) pressed_index = 2; // BTN2
38      else if (btn_state & 0x8) pressed_index = 3; // BTN3
39
40      // Wait and lock execution until the user lets go of the
        physical button
41      while (XGpio_DiscreteRead(&GpioBtns, GPIO_CHANNEL) != 0) {
42          usleep(10000);
43      }
44
45      return pressed_index;
46  }
47
48  /*
49   * Utility Function: Flashes all 4 board LEDs simultaneously.
50   * Used for game states like "Game Start Alert" or "Level
        Cleared Success".
51   */
52  void flash_all_leds(int count, int delay_ms) {
53      for (int i = 0; i < count; i++) {
54          XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL, 0xF); //
                Turn all 4 LEDs ON
55          usleep(delay_ms * 1000);
56          XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL, 0x0); //
                Turn all 4 LEDs OFF
57          usleep(delay_ms * 1000);

```

```

58     }
59 }
60
61 int main() {
62     int sequence[MAX_ROUND];
63     int current_score = 0;
64
65     // Seed the randomizer loop with an arbitrary number
66     srand(54321);
67
68     // Step 1: Initialize software links to hardware registers
69     if (XGpio_Initialize(&GpioLeds, LED_GPIO_DEVICE_ID) !=
70         XST_SUCCESS) return XST_FAILURE;
71     if (XGpio_Initialize(&GpioSwitches, SWITCH_GPIO_DEVICE_ID)
72         != XST_SUCCESS) return XST_FAILURE;
73     if (XGpio_Initialize(&GpioBtns, BTN_GPIO_DEVICE_ID) !=
74         XST_SUCCESS) return XST_FAILURE;
75
76     // Step 2: Establish data bus directions (0 = Output, 0xF/1
77         = Input)
78     XGpio_SetDataDirection(&GpioLeds, GPIO_CHANNEL, 0x0); //
79         LEDs are Outputs
80     XGpio_SetDataDirection(&GpioSwitches, GPIO_CHANNEL, 0xF); //
81         Switches are Inputs
82     XGpio_SetDataDirection(&GpioBtns, GPIO_CHANNEL, 0xF); //
83         Buttons are Inputs
84
85     // Main persistent outer game loop
86     while (1) {
87         // Play Intro Alert: Flash all LEDs 3 times rapidly to
88             signal a new game
89         flash_all_leds(3, 150);
90         current_score = 0;
91         usleep(1000000); // 1-second pause before Level 1 starts
92
93         // Inner round tracker loop
94         while (current_score < MAX_ROUND) {
95             // A. Add a new random step (0 to 3) to the hidden
96                 memory sequence
97             sequence[current_score] = rand() % 4;
98             current_score++;

```

```

90
91 // B. Playback Phase: Show the sequence pattern to
92 the player
93 for (int i = 0; i < current_score; i++) {
94     // Left shift 1 by the sequence index to target
95     individual bit positions
96     XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL, (1
97         << sequence[i]));
98     usleep(500000); // Keep light illuminated for
99     half a second
100     XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL,
101         0x0);
102     usleep(200000); // Quick blank gap between
103     flashes
104 }
105
106 // C. Player Input Phase: Track and evaluate button
107 presses
108 int round_success = 1;
109 for (int i = 0; i < current_score; i++) {
110     // Wait until player registers a button press
111     int user_guess = get_pressed_button();
112
113     // Provide instant visual response by lighting
114     up the guessed LED briefly
115     XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL, (1
116         << user_guess));
117     usleep(200000);
118     XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL,
119         0x0);
120
121     // Check if player input matches the hidden
122     pattern
123     if (user_guess != sequence[i]) {
124         round_success = 0; // Set failure flag
125         break;             // Break user input check
126                             loop immediately
127     }
128 }
129
130 // D. Result Scoring Phase

```

```

119         if (round_success == 0) {
120             // Game Over: Light up all 4 LEDs solidly for 2
121                 seconds to show defeat
122             XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL,
123                 0xF);
124             usleep(2000000);
125             XGpio_DiscreteWrite(&GpioLeds, GPIO_CHANNEL,
126                 0x0);
127             usleep(500000);
128             break; // Break round tracker to reset the game
129                 entirely
130         } else {
131             // Round Cleared: Flash all LEDs twice quickly
132                 as a reward indicator
133             usleep(300000);
134             flash_all_leds(2, 100);
135             usleep(800000); // Brief pause before starting
136                 the next harder level
137         }
138     }
139     return 0;
140 }

```

Listing 6.1: Memory Game Application Source Code

Chapter 7

Results

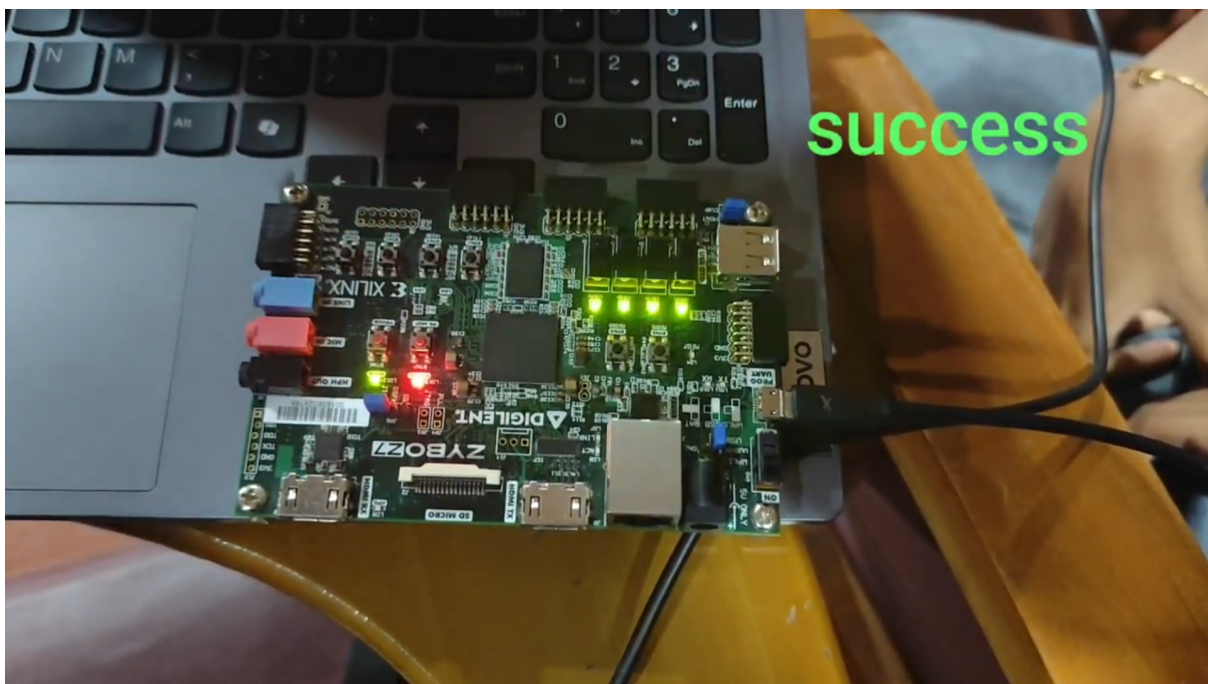


Figure 7.1: Memory Game Running on Zybo Z7-10 Board

The Memory Game was successfully implemented on the Zybo Z7-10 development board.

The system correctly:

- Generated random LED sequences.
- Displayed sequences through LEDs.
- Detected push button inputs.
- Verified user responses.
- Progressed through multiple levels.
- Indicated success and failure conditions.

The integration between the Processing System and Programmable Logic operated reliably through AXI GPIO interfaces.

Chapter 8

Applications

8.1 Educational Tool for Embedded Systems

Demonstrates hardware-software co-design using the Zynq platform.

8.2 Memory and Cognitive Training

Improves memory retention, concentration, and reaction speed.

8.3 FPGA-Based Interactive Systems

Serves as a foundation for developing interactive embedded applications.

8.4 Human-Machine Interface Research

Can be extended for advanced user interaction studies.

8.5 Embedded Gaming Systems

Provides a framework for developing FPGA-based games and entertainment systems.

Chapter 9

Conclusion

The Memory Game was successfully designed and implemented on the Zybo Z7-10 development board using the Zynq-7000 Processing System and Programmable Logic architecture. The project demonstrated effective integration between hardware and software using AXI GPIO peripherals and Embedded C programming.

The implementation provided hands-on experience in Vivado hardware design, Vitis software development, AXI communication, and embedded system debugging. The game operated as intended by generating random LED sequences, accepting user inputs through push buttons, and verifying responses in real time.

This project highlights the advantages of hardware-software co-design and demonstrates how Zynq-based systems can be used to develop interactive and educational embedded applications.

Chapter 10

References

1. AMD Xilinx, *Zybo Z7-10 Reference Manual*, Digilent Inc.
2. AMD Xilinx, *Vivado Design Suite User Guide*.
3. AMD Xilinx, *Vitis Unified Software Platform Documentation*.
4. Digilent Inc., *Zybo Z7 FPGA Development Board Documentation*.